# Softwareprojekt

## CampusKompass

Vom Bedarf zur Webanwendung

# Agenda

1. Ziel und Funktionen
2. User Stories und Use Cases
3. MoSCoW-Priorisierung
4. Architektur
5. Bedienung und Weboberfläche

# 1. Ziel und Funktionen

## Ausgangssituation

Studierende benötigen für einen Studientag Informationen aus mehreren Quellen. CampusKompass bündelt diese Informationen in einem lokalen Dashboard und ordnet sie dem ausgewählten Datum und Studienort zu.

## Ziel

Eine zentrale, einfach bedienbare Weboberfläche für die Tagesplanung an den Standorten Schmalkalden und Gera.

# 1. Ziel und Funktionen

## Kernfunktionen

- Studienplan nach Datum filtern
- Termine mit Uhrzeit, Veranstaltung, Lehrkraft, Ort und Raum anzeigen
- passenden Mensaplan laden
- Wetter für Datum, Uhrzeit und Campus anzeigen
- Campusziel für Route und ÖPNV-Fahrplan bestimmen
- Google-Maps-Route aus Startort oder Browserstandort öffnen
- Auswahl mit „Jetzt“ auf die aktuelle Zeit zurücksetzen

## Leitidee

Die zentrale Datums- und Zeitauswahl steuert alle abhängigen Ansichten.

# 2. User Stories und Use Cases

## Von Bedürfnissen zu Anforderungen

User Stories beschreiben den erwarteten Nutzen aus Sicht der Studierenden.
Use Cases konkretisieren anschließend den Ablauf, beteiligte Akteure, Vorbedingungen, Alternativen und Ergebnisse.

```
Nutzerbedarf
    ↓
User Story
    ↓
Use Case mit Ablauf und Akteuren
    ↓
Umsetzung im Code
```

# 2. User Stories und Use Cases

## User Stories

| ID | Nutzerziel | Use Case |
|---|---|---|
| CAMPUS-01 | Studienplan für ein Datum sehen | UC-02 |
| CAMPUS-02 | passenden Mensaplan sehen | UC-03 |
| CAMPUS-03 | Wetter am Campus sehen | UC-04 |
| CAMPUS-04 | Route zum Campus planen | UC-05 |
| CAMPUS-05 | Fahrplan für Datum und Uhrzeit öffnen | UC-06 |
| CAMPUS-06 | Auswahl auf „Jetzt“ setzen | UC-07 |

Beispiel: „Als Student möchte ich den Studienplan für ein Datum sehen, damit ich meine Termine kenne.“

# 2. User Stories und Use Cases

## Beispiel: UC-02 Studienplan anzeigen

1. Student/in wählt ein Datum.
2. Frontend filtert `study-plan.json`.
3. Termine werden nach Uhrzeit sortiert.
4. Veranstaltung, Lehrkraft, Ort und Raum werden angezeigt.
5. Der Studienort bestimmt Mensa, Wetterstandort und Routenziel.

**Alternative:** Für einen Tag ohne Termine wird ein Hinweis angezeigt.

Die Use-Case-ID verbindet die fachliche Anforderung mit der späteren technischen Umsetzung.

# 3. MoSCoW-Priorisierung

## Warum priorisieren?

Nicht jede gewünschte Funktion kann gleichzeitig umgesetzt werden. MoSCoW hilft, den MVP auf den wichtigsten Nutzen zu konzentrieren.

- **Must have:** unverzichtbar für den Kernnutzen
- **Should have:** wichtig, aber notfalls verschiebbar
- **Could have:** sinnvoller Zusatz
- **Won't have:** bewusst nicht Bestandteil dieser Version

# 3. MoSCoW-Priorisierung

| Priorität | CampusKompass-Anforderungen |
|---|---|
| **Must have** | Studienplan für ein Datum anzeigen |
| **Should have** | Mensaplan und Wetter am passenden Campus |
| **Could have** | Campusroute, Fahrplanauskunft und „Jetzt“-Funktion |
| **Won't have** | Funktionen außerhalb des priorisierten MVP-Umfangs |

Die Priorisierung beeinflusst anschließend Architektur, Implementierungsumfang und Reihenfolge der Entwicklung.

# 3. MoSCoW-Priorisierung

## Rückverfolgbarkeit

```
CAMPUS-01 → UC-02 → study-plan.json → Studienplan-Karte
CAMPUS-02 → UC-03 → /api/mensa → Mensa-Karte
CAMPUS-03 → UC-04 → Open-Meteo → Wetter-Karte
CAMPUS-04 → UC-05 → Google Maps → Routen-Karte
CAMPUS-05 → UC-06 → VMT → Fahrplan-Widget
CAMPUS-06 → UC-07 → Datums-/Zeitsteuerung
```

So bleibt nachvollziehbar, welche Funktion aus welcher Anforderung entstanden ist und wo sie im System umgesetzt wird.

# 4. Architektur

## Systemaufteilung

```
Browser
   ├─ index.html + css/style.css
   ├─ js/app.js
   ├─ js/campus-context.js
   └─ study-plan.json
            │
            └── server.py
                  └── /api/mensa
                        └── Studierendenwerk Thüringen
```

Der Browser übernimmt Darstellung und Interaktion. Der Python-Server liefert statische Dateien aus und vermittelt ausschließlich den Mensa-Aufruf.

# 4. Architektur

## Zuständigkeiten

| Baustein | Verantwortung |
| --- | --- |
| `index.html` | Struktur und Eingabefelder der Oberfläche |
| `css/style.css` | Layout, responsive Darstellung und Zustände |
| `js/app.js` | Laden, Validieren, Filtern und Rendern der Daten |
| `js/campus-context.js` | Campus-, Mensa-, Wetter- und Routenzuordnung |
| `study-plan.json` | strukturierte Studienplandaten |
| `server.py` | lokaler Server und validierter Mensa-Proxy |

Die Trennung reduziert die Auswirkungen von Änderungen und macht die Komponenten einzeln überprüfbar.

# 4. Architektur

## Campusabhängiger Kontext

Aus dem Studienort werden zentrale Folgeinformationen abgeleitet:

```
Studienplan + ausgewähltes Datum
                ↓
          Campus-Kontext
        ↙       ↓        ↘
    Mensa    Wetter    Routenziel
                ↓
          Fahrplanziel
```

Die Zuordnung ist in `campus-context.js` gebündelt. Neue Standorte können dadurch zentral ergänzt werden.

# 4. Architektur

## Klassendiagramm Frontend mit PlantUML

Quelle: `modellierung/klassendiagramm_frontend.puml`

```
class CampusDashboard {
  +loadStudyPlan()
  +renderStudyPlan(selectedDate)
  +loadMenu()
  +loadWeather()
  +showRoute(origin)
}

class StudySession
class Mensa
class Meal
class WeatherData
class RouteDestination

CampusDashboard "1" o-- "*" StudySession
```

# 4. Architektur

## Klassendiagramm Backend mit PlantUML

Quelle: `modellierung/klassendiagramm_backend.puml`

```
class DashboardHandler {
  +do_GET()
}

class SimpleHTTPRequestHandler
class ThreadingHTTPServer
class Request

DashboardHandler --|> SimpleHTTPRequestHandler
ThreadingHTTPServer o-- DashboardHandler
DashboardHandler ..> Request
DashboardHandler ..> "Studierendenwerk Thüringen"
```

Das Diagramm zeigt, dass `DashboardHandler` den Standard-HTTP-Handler

# 4. Architektur

## Wofür werden die Klassendiagramme genutzt?

- **Entwurf:** Zuständigkeiten werden vor bzw. während der Implementierung sichtbar gemacht.
- **Kommunikation:** Teammitglieder können über dieselben Begriffe sprechen.
- **Rückverfolgbarkeit:** Use Cases lassen sich auf Datenobjekte und Funktionen im Code beziehen.
- **Wartung:** Änderungen an Campus-Kontext, Frontend oder Proxy können gezielter eingeordnet werden.
- **Dokumentation:** Die `.puml`-Dateien bleiben versionierbare Quellen und können mit PlantUML als Diagramme gerendert werden.

Die Diagramme sind damit Entwicklungs- und Dokumentationshilfen, keine zusätzliche Programmlogik.

# 4. Architektur

## Umsetzung der Anforderungen im Code

1. **Datenbasis:** Termine liegen strukturiert in `study-plan.json`.
2. **Auswahl:** Datum und Uhrzeit werden zentral verwaltet.
3. **Verarbeitung:** `app.js` filtert Termine und bestimmt den Campus.
4. **Datenzugriff:** Wetter, Karten und Fahrplan werden über externe Dienste geladen; die Mensa läuft über `/api/mensa`.
5. **Darstellung:** Ergebnisse werden in separaten Dashboard-Karten gerendert.

Fehler bei JSON-Daten, Netzwerkzugriffen oder API-Antworten werden als Statusmeldungen in der Oberfläche angezeigt.

# 4. Architektur

## Warum ein Mensa-Proxy?

Der Browser darf den externen Mensa-Endpunkt nicht zuverlässig direkt ansprechen, weil der Dienst keinen passenden CORS-Zugriff bereitstellt.

```
Browser → server.py → Studierendenwerk Thüringen
          Validierung       Speiseplan
```

`server.py` akzeptiert nur bekannte Ressourcen-IDs und gültige Datumswerte. Fehler werden ausdrücklich als HTTP 400 oder 502 zurückgemeldet.

# 5. Bedienung und Weboberfläche

## Start und erster Ablauf

1. Server mit `python .\server.py` starten.
2. `http://127.0.0.1:8000/` im Browser öffnen.
3. Datum und Abfahrtszeit in der Tagesplanung auswählen.
4. Studienplan, Mensa und Wetter prüfen.
5. Route oder Fahrplan für den angezeigten Campus verwenden.

Die zentrale Auswahl aktualisiert alle abhängigen Karten.

# 5. Bedienung und Weboberfläche

## Oberfläche

- Kopfbereich mit Logos und aktueller Zeit
- zentrale Tagesplanung für Datum und Abfahrtszeit
- Studienplan-Karte mit den Terminen des Tages
- Mensa- und Wetter-Karten
- Fahrplan-Widget und Karte für den Weg zum Campus
- sichtbare Status- und Fehlermeldungen
- responsive Darstellung für Desktop und mobile Geräte

Die Oberfläche verwendet beschriftete Formulare, sichtbare Fokuszustände und `aria-live`-Bereiche für dynamische Ergebnisse.

# 5. Bedienung und Weboberfläche

## Beispiel: Campuswechsel

1. Das ausgewählte Datum enthält einen Termin in Gera.
2. Der Studienplan liefert den Studienort.
3. `campus-context.js` wählt Gera als Kontext.
4. Mensa, Wetter, Google Maps und VMT erhalten die Gera-Ziele.
5. Die Karten zeigen die aktualisierten Informationen.

Dadurch muss die Nutzerin oder der Nutzer den Campus nicht in mehreren Bereichen manuell einstellen.

# 5. Bedienung und Weboberfläche

## Entwicklungs- und Qualitätsgedanke

Der Weg vom Bedarf zum Ergebnis bleibt überprüfbar:

```
User Story
   → Use Case
      → MoSCoW-Priorität
         → Architekturentscheidung
            → Code und Daten
               → Bedienbare Weboberfläche
```

Der CampusKompass zeigt damit nicht nur ein Ergebnis, sondern den vollständigen Software-Engineering-Prozess hinter der Anwendung.

# Fazit

- CampusKompass bündelt studienbezogene Informationen in einer Oberfläche.
- Anforderungen werden über User Stories und Use Cases konkretisiert.
- MoSCoW priorisiert den MVP und begrenzt den Entwicklungsumfang.
- Die Architektur trennt Daten, Campuslogik, Oberfläche und Proxy.
- Die Umsetzung endet in einer direkt nutzbaren Weboberfläche.